AI PLACEMENT OS

UNIVERSAL REALTIME VOICE AGENT

Locked Product Specification and Development Roadmap

LOCKED PRODUCT DEFINITION

A realtime, user-scoped AI career agent that knows the authenticated user's Placement OS, understands the live external career world, and can safely operate supported app capabilities through validated tools.

SCOPE RULE

Feature development first. Signing, production backend deployment, publishing, and Play Store submission remain paused until the feature program is finished and the final audit is passed.

Prepared as the implementation source of truth

Current project baseline: ai-placement-android + ai-placement-backend

Document map

Section	Purpose
1. Locked product definition	What the agent is, what it knows, and what it can do.
2. Capability map	Personal data, world intelligence, app control, voice behavior and safety.
3. Technical architecture	Client, realtime session, agent brain, tool layer and backend boundaries.
4. Development process	Exact staged build sequence and gates from audit to acceptance.
5. Detailed stages VA-0 to VA-12	Scope, implementation, tests, outputs and stop conditions.
6. Quality and security program	Reliability, concurrency, prompt injection, privacy and evaluation.
7. Provider and cost strategy	How provider choices stay swappable and ₹0 during feature development.
8. Final acceptance	What must be true before this feature is considered complete.
9. Research basis	Current external architecture/policy references used to shape the plan.

> **IMPLEMENTATION RULE**
>
> Do not start implementation from memory or from this document alone. VA-0 must first inspect the actual current code and create the concrete file/API map for the selected branch state.

1. Locked product definition

The Voice Agent is the universal conversational interface to the AI Placement OS. It is not a separate chatbot and not merely a voice wrapper around one LLM. The agent sits above the existing domain services and becomes a safe control plane for reading career information, retrieving live market intelligence, navigating the app, and executing supported actions.

1.1 The three knowledge planes

Plane	What it contains	Why it matters
Personal Placement OS	Profile, resume, ATS, skills, projects, GitHub/professional presence, Career Twin, goals, paths, strategy, jobs, applications, interviews, assessments, weaknesses, preparation, missions, progress and outcomes.	Lets the agent answer questions about the individual user.
Live career world	Current jobs, company hiring, role requirements, skill demand, technology trends, market signals, geography, current learning resources and other fresh career information.	Prevents stale answers about a changing job market.
App capabilities	Navigation and supported read/write workflows exposed through a controlled tool registry.	Lets the user use the app by speaking rather than manually operating every screen.

> **CORE PROMISE**
>
> Anything the authenticated user can legitimately and safely do inside the Placement OS should be expressible as a voice interaction where the application actually supports that capability. The agent does not gain unrestricted control of the phone or other apps.

1.2 Example end-to-end interaction

User: "Find the best Data Engineer jobs for me right now.

Agent: 1) retrieve my target profile and evidence
2) search fresh job sources
3) normalize/deduplicate jobs
4) compare requirements to my evidence
5) rank the matches
6) explain the recommendation

User: "Prepare me for the best one."
Agent: 1) identify the selected job
2) inspect gaps and past interview weaknesses
3) create or retrieve preparation
4) begin the relevant preparation/interview flow

2. Complete capability map

2.1 What the agent can know about the user

- Profile: identity, education, experience and user-approved preferences.
- Resume and ATS: parsed content, analyses, issues, recommendations and history.
- Skills and evidence: skills, gaps, passport, evidence and role alignment.
- Projects and presence: projects, proof, GitHub evidence and professional presence.
- Career intelligence: Career Twin, Career Goal, Career Path, Career Strategy and Career Decision data.
- Jobs and applications: listings, saved jobs, matches, saved searches, applications, notes, outcomes and follow-ups.
- Interview and assessment intelligence: interview history, answers, evaluations, recruiter simulations, assessments and weaknesses.
- Execution systems: roadmap, preparation plans, missions, Next Best Action, progress, Placement Path and relevant notifications.

2.2 What the agent can know about the outside world

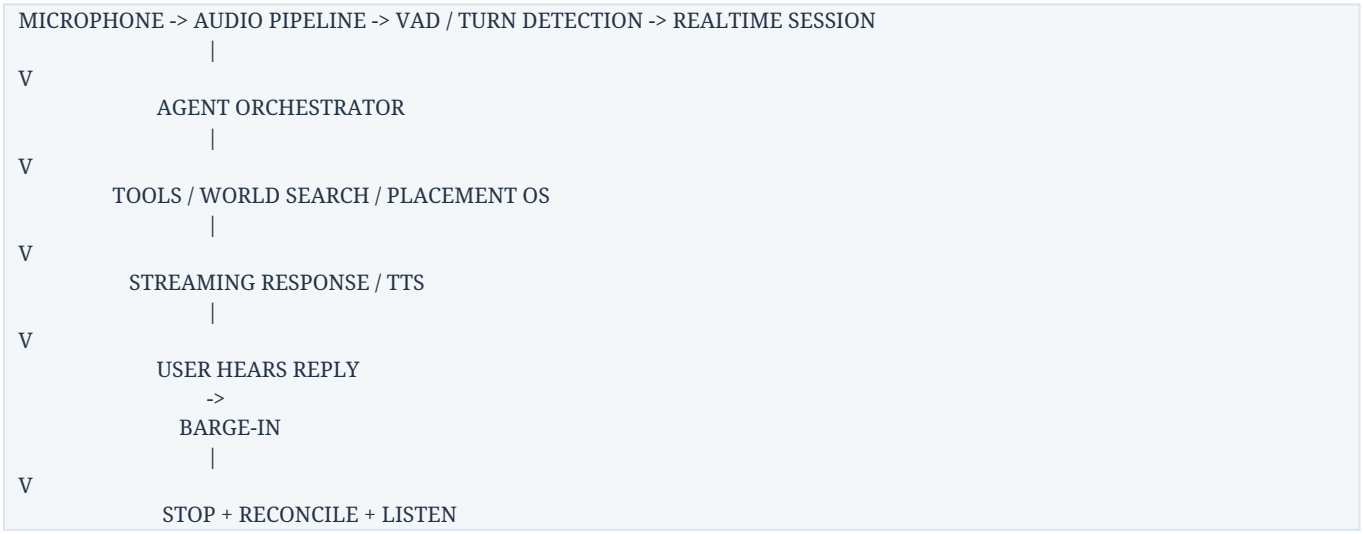
- Current job openings and role requirements.
- Current company hiring information from allowed sources.
- Skill-demand and technology-adoption signals.
- Current technology and role trends.
- Geographic career markets, where reliable data exists.
- Learning resources and current documentation.
- Publicly available interview/topic information where permitted.
- Source, retrieval time, geography, freshness and confidence metadata.

2.3 What the agent can do inside the app

Action class	Examples	Default control
Read	Read Career Twin, weaknesses, applications, scores, saved jobs, history, progress.	Execute after authorization; no confirmation for ordinary reads.
Safe navigation	Open ATS, open application, open weaknesses, show progress.	Execute directly.
Moderate mutation	Save job, add note, create preparation, start interview, create reminder.	Validate parameters; reuse existing idempotency/concurrency.
High-impact mutation	Delete data, materially change outcomes, submit a consequential action.	Explicit confirmation, ideally visible + spoken, then execute and verify.

> **NON-NEGOTIABLE**
>
> The model is never the authorization boundary. The authenticated backend and domain services remain authoritative for ownership, permission, business rules and mutation safety.

3. Realtime voice experience



3.1 Required realtime behaviors

- Streaming conversation rather than the core agent being limited to record-and-upload files.
- Voice activity detection plus turn detection so pauses, backchannels and real interruptions are handled differently.
- Barge-in: user speech immediately interrupts agent playback and the session state is reconciled.
- False-interruption protection for backchannels such as "okay" or "mm-hmm".
- Preemptive response preparation when confidence is sufficient, with cancellation if the user continues speaking.
- Audio focus, echo handling, noise suppression, gain control and device routing.
- Session reconnect/resume with conversation state and operation state preserved.
- Cancellation propagation from voice playback through model generation and safely cancellable tool work.
- Graceful degradation to push-to-talk or text if realtime voice becomes unavailable.

3.2 Voice session state machine

```
IDLE
  |
V
LISTENING <-> INTERRUPTED
  |
V
TRANSCRIBING / TURN CONFIRMED
  |
V
THINKING
  |
V
TOOL_CALLING (zero or more)
  |
V
GENERATING
  |
V
SPEAKING
  |
V
LISTENING

Error states: PERMISSION_ERROR / AUDIO_ERROR / NETWORK_ERROR / TOOL_ERROR / AI_ERROR / SESSION_EXPIRED
```

3.3 Current repository foundation to reuse

The Android repository already has VoiceInterviewScreen, VoiceInterviewViewModel, VoiceInterviewManager, AudioRecorder and AudioPlayer, plus interview API/repository support. The backend already has interview audio routes and a provider path using faster-whisper and pyttsx3 for development-oriented voice processing. The existing voice-readiness checklist also identified audio-focus handling and configuration/process-death robustness as concrete gaps to resolve in the shared Voice Core rather than copy forward.

> **DESIGN DECISION**
>
> Create a shared Voice Core first. The existing voice interview feature becomes one consumer/mode of the new Voice Core; we do not create a second independent audio stack.

4. Technical architecture



4.1 Tool registry contract

Field	Requirement
Capability name	Stable, human-meaningful identifier for one supported capability.
Read/write class	Explicitly marks whether the capability only reads or changes state.
Inputs	Typed, validated arguments. The LLM does not bypass schemas.
Ownership	Server derives authenticated user context; caller does not choose another user/profile.
Permission	Required user permission or product entitlement when applicable.
Confirmation	Whether the operation requires explicit user confirmation.
Side effects	Documented state changes and external effects.
Idempotency	Existing operation-claim infrastructure reused where required.
Timeout/cancellation	Defined behavior for long or cancelled calls.
Result verification	Agent may claim success only after authoritative success.

4.2 Personal tool examples

```
get_profile()
get_career_twin()
get_skill_gaps()
get_resume_analysis()
get_saved_jobs()
get_job_match(job_id)
get_application(application_id)
get_interview_history()
get_weaknesses()
get_preparation_plan(job_id)
get_daily_mission()
get_next_best_action()
get_progress()
```

4.3 World tool examples

```
search_live_jobs(query, location, filters)
get_company_jobs(company)
get_role_requirements(role)
get_skill_demand(skill, geography)
get_technology_trends(topic, geography)
compare_markets(role, geographies)
search_learning_resources(topic, level)
search_official_documentation(topic, version)
```

5. Agent intelligence and knowledge grounding

5.1 Personal context retrieval

The agent should retrieve the smallest relevant set of user data needed for the current request. It must not inject the entire Placement OS into every model call. The retrieval plan should be driven by intent and current conversation context.

5.2 World intelligence freshness

Data type	Freshness policy	Answer rule
Live job openings	Very short freshness window; refresh for current queries.	State that the count/results are from the latest retrieval.
Company hiring	Short freshness window and source provenance.	Prefer official company sources where available.
Market/technology trend	Short/medium window depending on source.	Use dated evidence and avoid timeless claims.
Technical documentation	Version-aware and source-aware.	Prefer official documentation for API behavior.
Learning resources	Refresh when current availability matters.	Distinguish recommendation from authoritative course/provider data.

5.3 Evidence language

Evidence state	Agent language pattern
Known	"Your stored assessment shows..."
Live observed	"The latest job search returned..."
Derived	"Based on those requirements and your evidence, I estimate..."
Recommended	"I recommend..."
Unknown	"I do not have enough reliable evidence to answer that confidently."

5.4 World-source hierarchy

- Official company/government source when directly available.
- Authoritative research or labor-market sources.
- Trusted job/market data providers with provenance.
- Reputable publications.
- Community/user-generated information only as secondary evidence and clearly labeled.

5.5 Job normalization

The world layer must normalize titles, companies, locations, requirements and identifiers, then deduplicate equivalent postings from multiple sources before exposing counts or rankings. Existing job-hunt normalization, deduplication and provenance components should be extended rather than duplicated.

6. Universal app control model

6.1 Command resolution

```
USER SPEECH
  |
V
Intent + entity extraction
  |
V
Use current-screen context when safe
  |
V
Resolve ambiguity only when needed
  |
V
Select capability
  |
V
Validate tool arguments
  |
V
Authorization / confirmation
  |
V
Execute existing domain service
  |
V
Verify authoritative result
  |
V
Update conversation + UI
  |
V
Speak result
```

6.2 Current-screen context

The Android client may provide ephemeral context such as current screen, selected job, selected application or current interview. This context is session-scoped and should not automatically become permanent memory.

6.3 Multi-step workflows

The agent must support plan -> execute -> verify for multi-step requests. Example: find relevant job -> compare -> identify gaps -> create preparation -> start interview. Each step is independently authorized and verified. If one step fails, the agent reports the exact state instead of claiming the entire plan succeeded.

6.4 Undo and dry run

Reversible operations should expose undo when business semantics allow it. The agent should support a dry-run intent such as "What would you do?" so users can inspect a proposed sequence before any state-changing operation.

6.5 Long-running operations

Operations such as deep analyses, preparation generation or broad job scans should expose progress and asynchronous completion where practical. The voice session must not block indefinitely on one tool; the agent can continue the conversation while the operation completes and then reconcile the verified result.

> **ANTI-HALLUCINATION RULE**
>
> Never speak "done" because a tool call was requested. Speak success only after the backend returns an authoritative success result. Speak failure with the actual verified state when the operation fails.

7. Memory, privacy and identity

7.1 Memory layers

Layer	Examples	Retention principle
Turn context	Current question, current job, current application, pending tool.	Ephemeral; clear on session end unless needed for an active operation.
Conversation summary	Recent career discussion and unresolved topic.	Bounded and compressed; retain only what improves follow-up.
User-approved memory	Target role, preferences, learning style, chosen goals.	Explicit, editable, deletable and user-scoped.
Placement OS data	Actual profile/application/career records.	Remain authoritative in existing domain storage; memory does not replace them.

7.2 Identity and secret boundaries

- The authenticated session determines the user; the LLM never chooses an arbitrary profile ID.
- No OAuth access tokens, passwords, API keys, signing keys or database credentials are exposed to the model.
- Sensitive voice data is minimized; raw audio should not be retained unless a feature explicitly requires it.
- External content is treated as untrusted data, never as agent instructions.
- Logs should record safe operational metadata rather than secrets or unnecessary raw sensitive content.

7.3 Permission model

Microphone, calendar, files, notifications and other device capabilities must be requested only when a current feature needs them and should be explained in context. The Play Store data-safety/privacy surface must match the implemented behavior. Current Google Play policy treats microphone and other personal/sensitive data as data requiring careful handling, disclosure and policy-conforming use.

7.4 Mobile realtime authentication

The preferred mobile architecture keeps the long-lived provider/API key server-side and uses short-lived client session credentials where the chosen realtime provider supports them. This keeps a master credential out of the APK.

8. Development process - master rules

8.1 Build in controlled stages

- One stage at a time. Do not start the next stage until the previous stage has passed its gate.
- Every stage begins with a read-only baseline of current files, branches, tests and behavior relevant to that stage.
- Use the existing architecture before introducing new abstractions. New abstractions must remove duplication or enforce a needed invariant.
- No unrelated refactor, dependency upgrade, UI redesign or backend cleanup.
- No deployment/publishing work during VA stages.
- No commit, push, reset, rebase, amend, force-push, stash, clean or branch switch unless explicitly requested later.

8.2 Required proof at each stage

Proof class	Minimum expectation
Functional	Happy paths for the capabilities introduced.
Negative	Invalid input, permission denial, unavailable provider and expected failure modes.
Security	Ownership, authorization, prompt injection and secret exposure checks where applicable.
Concurrency	Overlapping requests/actions do not duplicate or corrupt state where applicable.
Lifecycle	Rotation, process death, cancellation and reconnect where applicable.
Mutation	Deliberately remove key guardrails and prove tests fail, then restore and prove green.
Integration	Real HTTP/backend boundary for networked behavior; mocks alone are insufficient for critical paths.
Device	Real Android device validation for microphone, audio routing, interruption and lifecycle behavior.

8.3 Stage stop conditions

- Any P0 safety/ownership defect remains open.
- A critical action can be duplicated under retry or concurrency.
- The agent can claim a successful mutation without verified success.
- A critical realtime path has an untested lifecycle transition.
- An external-world answer has no freshness/provenance strategy where freshness matters.
- A provider failure causes the entire Placement OS to become unusable.

9. Development stages - overview

Stage	Name	Primary outcome
VA-0	Deep repository audit	Concrete architecture/file/API map; baseline; gaps; implementation contract.
VA-1	Shared Voice Core	Unified audio/session lifecycle layer shared by voice features.
VA-2	Realtime transport	Bidirectional realtime session with interruption, VAD/turn handling and reconnection.
VA-3	STT/TTS provider layer	Provider-neutral speech interfaces, fallback and latency/error policy.
VA-4	Placement OS tool registry	Safe read access to authenticated user data across existing domains.
VA-5	World Career Intelligence	Realtime jobs/companies/roles/technology/market/learning retrieval with provenance.
VA-6	Agent reasoning + memory	Intent, retrieval planning, grounded reasoning, context and bounded memory.
VA-7	Safe app actions	Navigation and supported mutations using confirmation, validation and existing idempotency.
VA-8	Voice Career Coach	Natural career coaching across personal + world context.
VA-9	Voice Interview Coach	Premium conversational interview mode built on the existing interview engine.
VA-10	Security + reliability hardening	Red-team, lifecycle, concurrency, tool safety and failure-injection closure.
VA-11	Real-device validation	Phone-based audio, device routing, noise, background/foreground and network matrix.
VA-12	Final Voice Agent audit	Full regression, evidence review, open-finding closure and acceptance decision.

ORDER MATTERS

The tool layer and security boundaries are established before broad action capabilities. World intelligence is connected before we claim the agent can answer current-market questions. Real device validation happens before production-readiness claims.

10. Detailed stages VA-0 to VA-3

VA-0 - Deep repository audit

Read-only. Inspect both current working trees and the exact branches in use. Inventory voice/audio code, network layer, auth/session handling, all domain APIs/repositories, existing AI provider abstraction, current job-hunt providers, current prompt/output guards, tests, navigation and lifecycle patterns.

Deliverables: architecture map; exact files/classes to reuse; list of current voice gaps; endpoint/tool inventory; provider constraints; test inventory; proposed interfaces; implementation sequence; explicit non-goals.

Gate: no code changes. Acceptance requires that the implementation plan is based on the actual repository, not assumptions.

VA-1 - Shared Voice Core

Build a single lifecycle-safe Voice Core. It owns microphone permission state, audio session, focus, recorder/player coordination, cancellation, state transitions and cleanup. It must survive recomposition and configuration changes without orphaning resources.

Key tests: permission denial/retry; audio focus loss; recorder failure; player failure; cancellation; rotation/process recreation; double start/stop; cleanup; device route changes where testable.

VA-2 - Realtime transport

Introduce the realtime session transport behind an interface. Support streaming audio, connection lifecycle, VAD/turn events, interruption and reconnect/resume. Keep provider-specific details out of domain logic.

Key tests: connect/disconnect; reconnect; interruption during speech; user interruption during tool call; session expiry; duplicate event handling; backpressure; degraded network.

VA-3 - STT/TTS provider layer

Create provider-neutral STT/TTS interfaces. Adapt the current development voice path rather than deleting it. Document production-provider requirements separately. Provide latency, failure and fallback semantics.

Key tests: valid/invalid audio; transcript correctness fixtures; TTS failure; unsupported format; provider timeout; provider unavailable; fallback path; no raw audio leakage.

11. Detailed stages VA-4 to VA-7

VA-4 - Placement OS tool registry

Expose read capabilities over existing domain services. Start read-only to prove identity, scoping, result contracts and retrieval efficiency. Add tool-level schemas and ownership checks without letting the model access raw SQL or database sessions.

Tool family	Examples
Profile/evidence	profile, resume, ATS, skills, projects, GitHub presence
Career intelligence	Career Twin, Goal, Path, Strategy, Decision
Job hunt	jobs, saved jobs, matches, searches, applications, notes, outcomes
Readiness	interviews, assessments, evaluations, weaknesses, progress
Execution	missions, NBA, roadmaps, preparation, Placement Path

VA-5 - World Career Intelligence

Connect live external sources through adapter interfaces. Reuse existing job provider budget, normalization, deduplication and provenance patterns. Add role, company, skill and technology intelligence without scraping restricted sources.

Key outputs: source registry; freshness rules; retrieval cache policy; conflict resolution; geographic handling; provider failure behavior; cost/rate-budget controls.

VA-6 - Agent reasoning + memory

Implement intent classification, context planning, tool selection, grounded response construction, bounded conversation memory and user-approved long-term preferences. Include known/inferred/recommended/unknown response semantics.

Key tests: multi-turn references; wrong tool selection; stale context; prompt injection in job text; hallucinated data; context compression; memory deletion; ambiguous requests.

VA-7 - Safe app actions

Add navigation and mutations through the same tool registry. Use plan -> validate -> authorize/confirm -> execute -> verify. Reuse existing idempotency and concurrency infrastructure. Add undo where business semantics allow it.

Key tests: duplicate voice commands; concurrent commands; network replay; confirmation bypass attempt; unauthorized target; tool returns failure; action succeeds but response is lost; reconnect during mutation.

12. Detailed stages VA-8 to VA-12

VA-8 - Voice Career Coach

Turn the general agent into a high-value career coach. Cross-link personal evidence with live world intelligence. Examples: readiness explanation, role selection, skill-gap analysis, market opportunity discovery, 7/30/60/90-day planning and daily coaching.

Gate: answers are grounded, current-data questions trigger live retrieval, and recommendations clearly differ from stored facts.

VA-9 - Voice Interview Coach

Refactor/reuse the existing voice interview flow as a mode of the shared Voice Core. Support spoken questions, spoken answers, evaluation and follow-up conversation. Preserve existing answer semantics, concurrency protections and AI boundaries.

Gate: no duplicate answer/AI/transcription on replay; interruption and recovery work; evaluation remains honest when ungraded; interview state survives lifecycle events.

VA-10 - Security + reliability hardening

Run a dedicated adversarial program: cross-account isolation, tool injection, prompt injection, malicious job descriptions, repeated commands, action races, fake tool results, session drops, provider failure, oversized inputs, cancellation races and secret-leak scans.

Mutation testing must prove critical authorization/confirmation/idempotency/reconciliation guards are load-bearing.

VA-11 - Real-device validation

Use a physical Android device. Validate microphone permission, background/foreground transitions, rotation, phone calls, Bluetooth/wired headsets, noise, echo, playback route changes, audio focus, battery constraints, network loss and reconnection.

Gate: all critical device scenarios pass or have an explicit documented, accepted limitation; no production-quality claim from emulator-only testing.

VA-12 - Final Voice Agent audit

Fresh audit from scratch. Re-enumerate capabilities, tools, domain access, world sources, permissions, lifecycle paths, realtime behavior, security controls, performance metrics and tests. Verify no stage-specific work has regressed earlier Placement OS guarantees.

Final decision: COMPLETE only when all critical gates are green and all remaining findings are intentionally accepted by product decision, not simply ignored.

13. Quality engineering program

13.1 Test pyramid for the Voice Agent

```
UNIT TESTS
  intent, state machine, tool schemas, memory, freshness, policy
        |
V
SERVICE / API TESTS
  ownership, tool execution, provider adapters, world retrieval
        |
V
CONCURRENCY / MUTATION
  duplicate commands, races, replay, authorization guard removal
        |
V
REAL HTTP
  transport, auth, streaming/session boundaries where practical
        |
V
REAL DEVICE
  audio hardware, routing, interruption, lifecycle, network
        |
V
FINAL ADVERSARIAL AUDIT
```

13.2 Critical invariants

- One authenticated user can never retrieve another user's data through a voice tool.
- A model cannot directly mutate the database or bypass domain services.
- A repeated mutation cannot create a duplicate state when idempotency semantics require one result.
- A lost response can be reconciled from authoritative state without double execution.
- A rejected or unverified action is never described to the user as successful.
- A realtime disconnect cannot silently restart a destructive operation.
- A live-market answer that requires freshness never relies only on static model memory.
- Untrusted external text cannot override trusted agent instructions.
- Provider failure degrades gracefully and does not destroy core app usability.

13.3 Performance metrics

- Time from user stop to first spoken response.
- Time to first tool invocation and tool completion.
- Interruption latency.
- Reconnect time.
- Voice provider error rate.
- Tool success/failure rate.
- Duplicate-action rate.
- World-search freshness and cache hit rate.
- Average and tail latency for major career queries.

14. Security and privacy hardening

14.1 Threat model

Threat	Control
Cross-account data access	Authenticated server context + resource ownership checks on every tool.
Prompt injection in job/JD/GitHub text	Treat external content as data; never execute instructions from retrieved content.
Tool argument manipulation	Typed schemas + server validation outside the model.
Destructive action without consent	Explicit confirmation policy + UI confirmation for high-impact actions.
Replay/duplicate mutation	Existing idempotency/concurrency protections and operation-state reconciliation.
Provider credential leakage	Keep master credentials server-side; use short-lived session credentials where supported.
Audio privacy	Minimize raw audio retention; delete temporary artifacts when no longer needed.
Sensitive logging	Structured operational logs with secrets and unnecessary raw content excluded.
Stale external data	Freshness metadata, expiry policy and live refresh for current queries.

14.2 Android and Play constraints

The design assumes user-initiated visible voice sessions as the primary experience. It does not depend on unrestricted background microphone access. Current Android policy places restrictions on background starts for microphone-related foreground services, and Android 17 further hardens background audio interactions. The final implementation must validate against the project's target SDK at the time of release.

14.3 Data safety

Because the agent handles microphone/audio and career records, the final product must align implemented data flows with the Google Play Data safety section, in-app disclosures, privacy policy and user consent. No new sensitive permission should be added merely because the future agent might use it.

15. Provider and cost strategy

15.1 Provider abstraction

```
RealtimeTransportProvider
SpeechToTextProvider
TextToSpeechProvider
AgentModelProvider
WorldSearchProvider
JobProvider
MarketIntelligenceProvider
```

Business logic talks to interfaces. Provider-specific SDKs, URLs, credential handling and capabilities live behind adapters. This protects the Placement OS from vendor lock-in and lets us evaluate future providers without rewriting the agent.

15.2 ₹0 feature-development rule

- Do not introduce paid infrastructure solely to prototype the agent.
- Use existing development/local voice components where they are adequate for deterministic engineering tests.
- Keep production-grade provider selection as an explicit decision after interface and performance tests.
- Budget realtime and world-search usage so retries, interruptions and repeated commands cannot multiply cost unexpectedly.
- Do not put provider master keys in Android code.

15.3 Production provider decision gate

Before selecting a final production voice stack, compare: realtime protocol support, Android compatibility, interruption quality, session resumption, STT accuracy for technical vocabulary, TTS quality, latency, privacy terms, data retention, regional availability, rate limits, failure modes, total operating cost and licensing. The implementation stages do not assume a specific vendor unless VA-0 finds an existing project constraint that requires one.

16. How we will actually execute each stage

```
1. FREEZE SCOPE FOR THE STAGE
2. READ-ONLY REPOSITORY AUDIT
3. WRITE A BASELINE REPORT
4. IDENTIFY REUSE VS NEW CODE
5. IMPLEMENT MINIMUM CORRECT CHANGE
6. ADD FOCUSED UNIT/INTEGRATION TESTS
7. RUN NEGATIVE + CONCURRENCY + MUTATION TESTS
8. RUN REAL HTTP / DEVICE VALIDATION WHEN APPLICABLE
9. REVIEW DIFF + GIT STATUS
10. PRODUCE FINAL STAGE REPORT
11. ACCEPT STAGE
12. ONLY THEN MOVE TO NEXT STAGE
```

16.1 Per-stage final report template

- Status: COMPLETE / PARTIAL / BLOCKED.
- Baseline: branch, HEAD, dirty state, relevant test count.
- Scope: exactly what was intended.
- Files changed: production vs test/config.
- Behavior proof: what was observed before and after.
- Security proof: ownership, prompt injection, permission/secret checks.
- Concurrency/lifecycle proof when relevant.
- Mutation results.
- Real HTTP/device results when applicable.
- Known limitations and open findings.
- Explicit statement that deployment/publishing was not touched.
- Next stage recommendation only after acceptance.

16.2 Change discipline

All existing dirty work is presumed intentional. Never destroy or hide it to make the stage easier. No cleanup refactor is allowed just because a file looks messy. Every change must be directly justified by the stage objective.

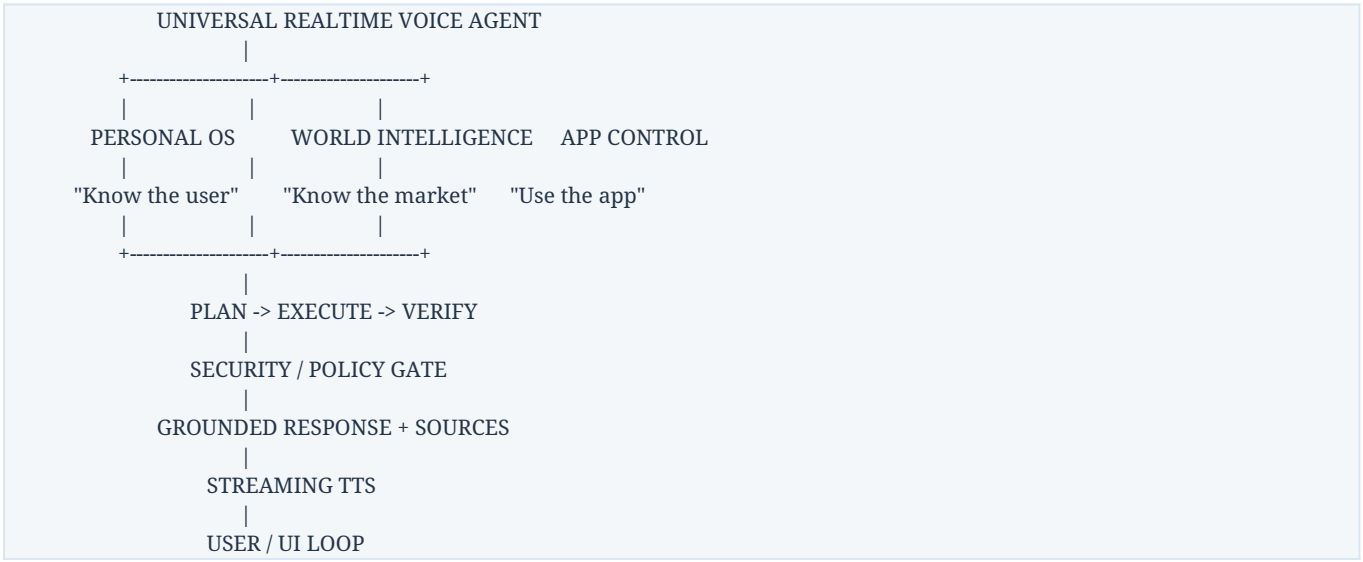
17. Final acceptance checklist

Area	Complete only when…
Realtime audio	Conversation is streaming, interruptible, reconnectable and lifecycle-safe.
Speech	STT/TTS are abstracted, tested, and production decision is explicit.
Personal data	All intended Placement OS domains are accessible through user-scoped tools.
World intelligence	Current questions trigger fresh retrieval; sources/freshness are preserved.
App control	Supported navigation/actions work through the tool registry with validation.
Actions	High-impact mutations require confirmation; every mutation is verified.
Concurrency	Duplicate/replayed/raced voice commands cannot corrupt state.
Memory	Conversation continuity works without unbounded or unintended retention.
Security	Cross-account, prompt-injection, secret, tool-abuse and destructive-action tests pass.
Privacy	Audio/data handling is minimized and documented; Play requirements are satisfied.
Reliability	Network loss, provider failure, cancellation, process death and session expiry are handled.
Device	Critical physical-device voice scenarios pass.
Testing	Unit, integration, mutation, real HTTP and device evidence exists for critical paths.
Regression	Existing Placement OS phases remain green.
Audit	VA-12 finds no unresolved P0/P1 defect.

DO NOT DECLARE COMPLETE EARLY

A green demo is not the acceptance criterion. The Voice Agent is complete only when behavior, security, reliability, freshness, lifecycle, device and regression evidence are all present.

18. Final product picture



The final experience should feel like talking to the Placement OS itself. The user can ask about their career, ask about the current job market, learn what to prepare for any role, discover current jobs and technologies, inspect their own progress, and operate supported application features. The agent combines private, user-scoped evidence with fresh world information and returns an answer that is explicit about what is known, what is inferred and what is recommended.

Deployment remains deferred

Signing, production backend deployment, production URL provenance, Play Store publishing and release submission are intentionally outside this development program. They resume only after the feature set is finished and VA-12 accepts the integrated system.

19. Research basis and repository sources

The development plan is grounded in the current project repositories plus current official platform/documentation references used during the research pass. These references inform the architecture; they do not replace the VA-0 audit of the exact working tree.

19.1 Project repositories

- https://github.com/sharu1234-coder/ai-placement-android
- https://github.com/sharu1234-coder/ai-placement-backend

19.2 Current project voice foundation inspected

- Android: VOICE_READINESS_CHECKLIST.md
- Android: app/src/main/java/com/aiplacementos/app/ui/interview/VoiceInterviewScreen.kt
- Android: app/src/main/java/com/aiplacementos/app/ui/interview/VoiceInterviewViewModel.kt
- Android: app/src/main/java/com/aiplacementos/app/voice/VoiceInterviewManager.kt
- Android: app/src/main/java/com/aiplacementos/app/voice/AudioRecorder.kt
- Android: app/src/main/java/com/aiplacementos/app/voice/AudioPlayer.kt
- Backend: app/api/interviews.py
- Backend: app/services/interview_service.py
- Backend: app/ai/ollama_provider.py
- Backend: requirements.txt

19.3 External current references

Reference	URL
OpenAI Realtime API overview	https://developers.openai.com/api/docs/guides/realtime
OpenAI Realtime client secrets	https://developers.openai.com/api/reference/resources/realtime/subresources/client_secrets/methods/create
LiveKit Agents	https://docs.livekit.io/agents/
LiveKit turn detection / interruption guidance	https://docs.livekit.io/agents/logic/turns/
Android background foreground-service restrictions	https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start
Android 17 background audio hardening	https://developer.android.com/about/versions/17/changes/bg-audio
Google Play Data Safety	https://support.google.com/googleplay/answer/11416267
Google Play User Data policy	https://support.google.com/googleplay/android-developer/answer/10144311

> **SOURCE DISCIPLINE**
>
> Current platform/provider facts can change. At implementation time, VA-0 and any provider-selection stage must re-check official documentation and current limits before coding against them.